# JDBC - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

**Java Full Stack Master Course**

# Table of Contents

# JDBC — MASTER NOTES

***A Complete Reference Book for Interviews, Production Work & System Understanding***

---

# TOPIC 1: JDBC ARCHITECTURE

---

## 1. Introduction

**What is it?** JDBC (Java Database Connectivity) is a standard Java API (`java.sql`/`javax.sql`) that provides a **uniform way to connect to and interact with relational databases** — regardless of which specific database vendor (MySQL, PostgreSQL, Oracle, SQL Server) is actually being used.

**Why introduced**: Before JDBC, Java code that needed to talk to a database would need vendor-specific, non-portable code for every different database — a program written for Oracle couldn't run against MySQL without rewriting all data-access code. JDBC introduces a standard set of interfaces (`Connection`, `Statement`, `ResultSet`) that EVERY compliant database vendor implements via their own **driver**, letting application code stay database-agnostic.

**Interview definition**: "JDBC is a Java API providing a standardized set of interfaces for connecting to, querying, and updating relational databases, with vendor-specific implementation details abstracted behind a pluggable driver architecture, allowing the same application code to work against different databases with minimal changes."

## 2. Real Life Analogy

- **Bank**: JDBC is like a universal ATM card format — any bank's ATM (database vendor) can process the SAME card format (JDBC API calls), because each bank has built its own machine (driver) that understands that universal card standard internally, even though each bank's actual internal systems are completely different.
- **Restaurant**: JDBC is like a standard order-ticket format used across an entire restaurant chain — each individual restaurant location (database vendor) has its OWN kitchen equipment (proprietary internal engine), but every kitchen has been built to understand the SAME standard ticket format.

## 3. The JDBC Architecture — Layers

```
+-------------------------------------------------------------+
| YOUR APPLICATION CODE |
| (writes to the STANDARD JDBC API - java.sql.*) |
+-------------------------------------------------------------+
|
v
+-------------------------------------------------------------+
| JDBC API (interfaces only) |
| Connection, Statement, PreparedStatement, ResultSet, etc. |
+-------------------------------------------------------------+
|
v
+-------------------------------------------------------------+
| JDBC DRIVER (vendor-specific IMPLEMENTATION) |
| e.g., mysql-connector-j.jar, postgresql.jar, ojdbc.jar |
+-------------------------------------------------------------+
|
v
+-------------------------------------------------------------+
| ACTUAL DATABASE SERVER |
| (MySQL, PostgreSQL, Oracle, SQL Server) |
+-------------------------------------------------------------+
```

Your application code only ever calls methods defined on the JDBC INTERFACES — the actual behavior is provided at runtime by whichever driver JAR is on the classpath for the specific database you're connecting to.

## 4. Two-Tier vs Three-Tier JDBC Models

| Model | Structure | Use Case |
|---|---|---|
| **Two-tier** | Application talks DIRECTLY to the database via a driver | Simple desktop apps, small internal tools |
| **Three-tier (typical enterprise)** | Application (via a middle-tier server, e.g., a Spring Boot service) talks to the database, with the ACTUAL client (browser/mobile app) talking only to that middle tier | Virtually all modern web/enterprise applications |

## 5. Code Example — The High-Level JDBC Workflow (Preview)

```
import java.sql.*;

public class JdbcArchitectureDemo {
public static void main(String[] args) throws SQLException {
String url = "jdbc:mysql://localhost:3306/company_db";
try (Connection conn = DriverManager.getConnection(url, "root", "password");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT name FROM employees")) {

while (rs.next()) {
System.out.println(rs.getString("name"));
}
}
// ALL FOUR core JDBC interfaces used here: Connection, Statement, ResultSet
// -> full detail on EACH is covered in the following dedicated topics
}
}
```

## 6. Common Mistakes

- Hardcoding vendor-specific SQL syntax deep in application logic assuming it will "just work" if the database is swapped — JDBC standardizes the CONNECTION/EXECUTION API, not the SQL dialect itself (SQL syntax differences between vendors still exist).
- Forgetting that the JDBC driver JAR must be on the classpath — without it, `DriverManager.getConnection()` fails, since there's no implementation available to actually connect.

## 7. Frequently Asked Interview Questions

1. **What is JDBC?** A standard Java API for connecting to and interacting with relational databases, with vendor-specific behavior provided by pluggable drivers.
2. **What problem does JDBC solve?** It lets Java application code remain database-vendor-agnostic, avoiding the need to rewrite data-access code for every different database.
3. **What are the four core JDBC interfaces you'll use in virtually every database interaction?** `Connection`, `Statement` (or `PreparedStatement`), `ResultSet`, and (implicitly) the `Driver`/`DriverManager`.
4. **Does JDBC standardize SQL syntax across all databases?** No — JDBC standardizes the CONNECTION and EXECUTION API; SQL dialect differences between vendors (e.g., `LIMIT` vs `TOP`) still exist and must be handled separately.

---

# TOPIC 2: DRIVERS

---

## 1. Introduction

**What is it?** A JDBC Driver is the vendor-specific implementation of the JDBC interfaces, translating standard JDBC API calls into the actual network protocol/communication a specific database understands.

## 2. Driver Types (Historical, Type 1-4)

| Type | Description | Modern Relevance |
|---|---|---|
| Type 1 (JDBC-ODBC Bridge) | Routes JDBC calls through an ODBC driver | REMOVED in Java 8+ — obsolete |
| Type 2 (Native-API) | Uses vendor-specific native (C/C++) client libraries | Rare today |
| Type 3 (Network Protocol) | Uses a middleware server to translate calls | Rare today |
| **Type 4 (Thin/Pure Java)** | Pure Java, talks DIRECTLY to the database over its native network protocol | The STANDARD, dominant driver type used today (e.g., MySQL Connector/J, PostgreSQL JDBC Driver) |

## 3. Loading a Driver and Connecting

```
// Modern approach (JDBC 4.0+, Java 6+): driver auto-registers via META-INF/services - NO
Class.forName() needed!
String url = "jdbc:postgresql://localhost:5432/mydb";
Connection conn = DriverManager.getConnection(url, "user", "password");

// OLD approach (pre-JDBC 4.0) - explicitly loading the driver class:
Class.forName("com.mysql.cj.jdbc.Driver"); // triggers the driver's static block to self-register
Connection conn2 = DriverManager.getConnection(url, "user", "password");
```

> Since JDBC 4.0 (Java 6+), drivers are discovered automatically via the **Service Provider Interface (SPI)** mechanism (a `META-INF/services/java.sql.Driver` file inside the driver JAR) — the explicit `Class.forName(...)` call is legacy and no longer required, though you'll still see it in older codebases and tutorials.

## 4. The JDBC URL Format

```
jdbc:<subprotocol>://<host>:<port>/<database>?<properties>

Examples:
jdbc:mysql://localhost:3306/company_db?useSSL=false
jdbc:postgresql://localhost:5432/company_db
jdbc:oracle:thin:@localhost:1521:orcl
```

## 5. Frequently Asked Interview Questions

1. **What is a JDBC Driver?** The vendor-specific implementation translating standard JDBC API calls into a specific database's actual communication protocol.
2. **Which driver type is standard/dominant today?** Type 4 (Thin/Pure Java) — communicates directly with the database over its native protocol, no native libraries or middleware needed.
3. **Do you still need `Class.forName("...")` to load a driver in modern Java?** No — since JDBC 4.0 (Java 6+), drivers self-register automatically via the Service Provider Interface mechanism when their JAR is on the classpath.
4. **What's the general structure of a JDBC URL?** `jdbc:<subprotocol>://<host>:<port>/<database>?<properties>`.

---

# TOPIC 3: CONNECTION

---

## 1. Introduction

**What is it?** `java.sql.Connection` represents a single, active session with a specific database — the starting point for every SQL operation (creating statements, managing transactions, retrieving metadata).

## 2. Obtaining and Using a Connection

```
import java.sql.*;

try (Connection conn = DriverManager.getConnection(
"jdbc:mysql://localhost:3306/company_db", "root", "password")) {

System.out.println("Connected: " + !conn.isClosed());
DatabaseMetaData meta = conn.getMetaData();
System.out.println("Database: " + meta.getDatabaseProductName());

} catch (SQLException e) {
e.printStackTrace();
}
```

## 3. Key `Connection` Methods

| Method | Purpose |
|---|---|
| `createStatement()` | Creates a `Statement` for executing plain SQL |
| `prepareStatement(sql)` | Creates a `PreparedStatement` (parameterized, safer, faster for repeated execution) |
| `prepareCall(sql)` | Creates a `CallableStatement` (for stored procedures) |
| `setAutoCommit(boolean)` | Controls whether each SQL statement auto-commits immediately, or requires an explicit `commit()` (see the Transactions topic) |
| `commit()` / `rollback()` | Finalize or undo a transaction |
| `close()` | Releases the database connection resource — CRITICAL to avoid connection leaks |
| `getMetaData()` | Returns database-level metadata (product name, version, supported features) |

## 4. Real Life Analogy

- **Bank**: A `Connection` is like an open, dedicated phone line to a specific bank branch — once established, you can issue multiple instructions (statements) over that SAME line, but the line itself is a finite, valuable resource that MUST be hung up (closed) when you're done, or the branch runs out of available lines for other customers.

## 5. Edge Cases

| Case | Result |
|---|---|
| Forgetting to `close()` a `Connection` | Connection LEAK — the underlying database connection resource is never released, eventually exhausting the connection pool/database's max connections |
| Using a `Connection` after it's been closed | Throws `SQLException` |
| Invalid credentials/unreachable database | `SQLException` thrown by `DriverManager.getConnection()` |

## 6. Common Mistakes

- Not using try-with-resources for `Connection` objects — leads to connection leaks if an exception occurs before a manual `close()` call is reached.
- Creating a brand-new `Connection` for every single query in a hot path instead of using a connection POOL (see the dedicated Connection Pooling topic) — connection creation is genuinely expensive (network handshake, authentication).

## 7. Best Practices

- Always use try-with-resources for `Connection`, `Statement`, and `ResultSet` — all three implement `AutoCloseable`.
- In production applications, NEVER call `DriverManager.getConnection()` directly per-request — use a connection pool (HikariCP, the modern industry standard) instead.

## 8. Frequently Asked Interview Questions

1 **What does `java.sql.Connection` represent?** An active session with a specific database, the starting point for creating statements and managing transactions.

2 **Why is it critical to always close a `Connection`?** Leaving it open leaks a finite, expensive database resource, eventually exhausting available connections for the entire application.

3 **What does `setAutoCommit(false)` do?** Disables automatic per-statement commits, requiring an explicit `commit()` (or `rollback()`) to finalize a group of statements as a single transaction.

4 **Should you create a new `Connection` for every database query in a production application?** No — use a connection pool to reuse a managed set of connections, since creating a fresh connection is expensive (network handshake, authentication).

---

# TOPIC 4: STATEMENT

---

## 1. Introduction

**What is it?** `java.sql.Statement` is the simplest way to execute a plain, static SQL string against the database — no parameters, the full SQL text is sent as-is.

## 2. Key Methods

| Method | Purpose | Returns |
|---|---|---|
| `executeQuery(sql)` | For `SELECT` statements | `ResultSet` |
| `executeUpdate(sql)` | For `INSERT`/`UPDATE`/`DELETE`/DDL | `int` (number of rows affected) |
| `execute(sql)` | For SQL that might be either (rarely used directly) | `boolean` (true if a `ResultSet` was produced) |

## 3. Code Example

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement()) {

ResultSet rs = stmt.executeQuery("SELECT id, name FROM employees");
while (rs.next()) {
System.out.println(rs.getInt("id") + ": " + rs.getString("name"));
}

int rowsUpdated = stmt.executeUpdate("UPDATE employees SET salary = salary * 1.1 WHERE department =
'IT'");
System.out.println(rowsUpdated + " rows updated");
}
```

## 4. THE Critical Danger — SQL Injection

```
// NEVER DO THIS - directly concatenating user input into SQL:
String userInput = "'; DROP TABLE employees; --";
String sql = "SELECT * FROM employees WHERE name = '" + userInput + "'";
stmt.executeQuery(sql); // DISASTER - the concatenated string can execute ARBITRARY SQL!
```

> **This exact vulnerability is why `PreparedStatement` (next topic) is the STANDARD, non-negotiable choice for any SQL involving user-supplied input in real production code.** Plain `Statement` should essentially NEVER be used with any value that originates from user input.

## 5. Common Mistakes

- Concatenating user input directly into a SQL string passed to `Statement` — the single most dangerous, common real-world security vulnerability in database-driven applications (SQL Injection).
- Using `Statement` for repeatedly-executed queries with varying values — loses the performance/caching benefits `PreparedStatement` provides (see next topic).

## 6. Frequently Asked Interview Questions

1. **What is `Statement` used for?** Executing plain, static SQL strings with no parameters.
2. **What critical security vulnerability does `Statement` expose you to if used carelessly with user input?** SQL Injection — an attacker can manipulate concatenated SQL strings to execute arbitrary, malicious SQL.
3. **What's the standard, correct alternative for any SQL involving user-supplied values?** `PreparedStatement`, which safely parameterizes input instead of directly concatenating it into the SQL string.

# TOPIC 5: PREPAREDSTATEMENT

## 1. Introduction

**What is it?** `PreparedStatement` extends `Statement`, representing a **precompiled** SQL statement with **placeholder parameters** (`?`), which are bound SAFELY (never directly concatenated into the SQL text) — the standard, mandatory choice for any SQL involving user-supplied or variable input.

**Why it matters (two distinct benefits)**:
1. **Security**: Parameters are sent SEPARATELY from the SQL command text to the database — the database driver/engine NEVER interprets bound parameter values AS SQL syntax, completely eliminating SQL Injection for properly parameterized queries.
2. **Performance**: The database can PARSE and PLAN the query ONCE, then reuse that compiled execution plan across many executions with different parameter values — significant for repeatedly-executed queries.

## 2. Code Example

```
import java.sql.*;

String sql = "SELECT * FROM employees WHERE department = ? AND salary > ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement pstmt = conn.prepareStatement(sql)) {

pstmt.setString(1, "IT"); // parameter index is 1-BASED, not 0-based!
pstmt.setDouble(2, 50000.0);

try (ResultSet rs = pstmt.executeQuery()) {
while (rs.next()) {
System.out.println(rs.getString("name"));
}
}
}
```

## 3. Why It's SQL-Injection-Safe (Internal Working)

```
Statement (UNSAFE):
SQL sent to DB: "SELECT * FROM employees WHERE name = 'Robert''; DROP TABLE employees; --'"
-> the ENTIRE string, including malicious injected SQL, is parsed AS SQL SYNTAX

PreparedStatement (SAFE):
SQL sent to DB (compiled ONCE): "SELECT * FROM employees WHERE name = ?"
Parameter sent SEPARATELY: "Robert'; DROP TABLE employees; --"
-> the parameter is NEVER parsed as SQL syntax at all - it's ALWAYS treated as
a literal DATA VALUE for that placeholder, no matter what characters it contains
```

## 4. Key Methods

| Method | Purpose |
|---|---|
| `setString(index, value)` / `setInt(index, value)` / etc. | Bind a typed parameter value (overloaded per type) |
| `setNull(index, sqlType)` | Explicitly bind a `NULL` value |
| `executeQuery()` | For `SELECT` (no SQL argument needed — it's already prepared) |
| `executeUpdate()` | For `INSERT`/`UPDATE`/`DELETE` |
| `addBatch()` / `executeBatch()` | Batch multiple parameter sets for efficient bulk execution (see the Batch Processing topic) |

## 5. `Statement` vs `PreparedStatement` — Comparison Table

| Aspect | `Statement` | `PreparedStatement` |
|---|---|---|
| SQL Injection risk | HIGH if user input is concatenated | ELIMINATED for properly parameterized values |
| Performance (repeated execution) | Recompiled every time | Compiled once, reused across executions |
| Readability with dynamic values | Messy string concatenation | Clean, explicit `?` placeholders |
| Recommendation | Only for truly static SQL with zero variable input | The DEFAULT choice for virtually all real-world SQL execution |

## 6. Edge Cases

| Case | Result |
|---|---|
| Forgetting to bind a required `?` parameter before executing | `SQLException` |
| Parameter index out of range (e.g., `setString(5, ...)` when only 2 `?` exist) | `SQLException` |
| Attempting to bind a table/column NAME (not a value) as a parameter | NOT possible — `PreparedStatement` parameters are for VALUES only, not identifiers (table/column names); dynamic identifiers require careful, separately-validated string construction (never from raw user input) |

## 7. Common Mistakes

- Forgetting parameter indices are 1-based, not 0-based.
- Trying to use a `?` placeholder for a table or column NAME — only works for VALUES, since the database needs to know the query structure at compile/plan time.
- Still occasionally concatenating SOME parts of a query (e.g., an `ORDER BY` column name derived from user input) directly into SQL, forgetting that identifiers require separate, careful allow-list validation since they can't be parameterized like values.

## 8. Best Practices

- Use `PreparedStatement` as the DEFAULT for essentially all SQL execution in production code, even for seemingly "static" queries — establishes a consistent, safe habit.
- Never construct dynamic table/column names from raw user input — validate against a strict allow-list if dynamic identifiers are genuinely needed.

## 9. Frequently Asked Interview Questions

1. **What's the main advantage of `PreparedStatement` over `Statement`?** It prevents SQL Injection by sending parameter values SEPARATELY from the SQL command text (never interpreted as SQL syntax), and can improve performance by reusing a precompiled execution plan.
2. **Why exactly does `PreparedStatement` prevent SQL Injection?** Because bound parameters are transmitted as pure DATA to the database, never merged into or parsed as part of the SQL command's structure/syntax, regardless of what characters they contain.
3. **Are parameter indices in `PreparedStatement` 0-based or 1-based?** 1-based.
4. **Can you use a `?` placeholder for a table or column name?** No — placeholders only work for VALUES; dynamic identifiers require separate, carefully allow-list-validated string construction.

---

# TOPIC 6: CALLABLESTATEMENT

---

## 1. Introduction

**What is it?** `CallableStatement` extends `PreparedStatement`, specifically designed to invoke **stored procedures** (precompiled SQL routines stored and executed directly within the database server itself).

## 2. Code Example

```
import java.sql.*;

// Assume a stored procedure exists: CALL GetEmployeeCount(IN dept VARCHAR(50), OUT total INT)
try (Connection conn = DriverManager.getConnection(url, user, pass);
CallableStatement cstmt = conn.prepareCall("{call GetEmployeeCount(?, ?)}")) {

cstmt.setString(1, "IT"); // IN parameter
cstmt.registerOutParameter(2, java.sql.Types.INTEGER); // OUT parameter - must register its TYPE

cstmt.execute();

int count = cstmt.getInt(2); // retrieve the OUT parameter's value AFTER execution
System.out.println("Employee count: " + count);
}
```

## 3. IN, OUT, and INOUT Parameters

| Parameter Type | Direction | Registration Needed |
|---|---|---|
| `IN` | Application → Database | `setXxx(index, value)`, same as `PreparedStatement` |
| `OUT` | Database → Application | `registerOutParameter(index, sqlType)` BEFORE execution, `getXxx(index)` AFTER |
| `INOUT` | Both directions | BOTH `setXxx()` before AND `registerOutParameter()` before, then `getXxx()` after |

## 4. Real Life Analogy

- **Bank**: A stored procedure is like a pre-approved, standardized banking FORM (e.g., "Process Loan Application") that the bank's OWN internal systems execute directly — you (the application) fill in the required input fields (IN parameters) and submit it, and the bank hands back specific result fields (OUT parameters), without you needing to know the internal multi-step processing logic executed on the bank's side.

## 5. `Statement` vs `PreparedStatement` vs `CallableStatement` — Full Comparison

| Aspect | `Statement` | `PreparedStatement` | `CallableStatement` |
|---|---|---|---|
| Purpose | Plain, static SQL | Parameterized SQL | Invoking stored procedures |
| Parameters | None (string concatenation only) | `IN` only (?) | `IN`, `OUT`, `INOUT` |
| Precompiled | No | Yes | Yes (the stored procedure itself is precompiled in the database) |

## 6. Frequently Asked Interview Questions

1. **What is `CallableStatement` used for?** Invoking database stored procedures, supporting `IN`, `OUT`, and `INOUT` parameters.
2. **What must you do before retrieving an `OUT` parameter's value?** Call `registerOutParameter(index, sqlType)` BEFORE executing the statement, then retrieve the value with the appropriate `getXxx()` method AFTER execution.
3. **What's the relationship between `CallableStatement` and `PreparedStatement`?** `CallableStatement` extends `PreparedStatement`, adding support for `OUT`/`INOUT` parameters specific to stored procedure calls.

---

# TOPIC 7: RESULTSET

---

# 1. Introduction

**What is it?** `ResultSet` represents the tabular result of a `SELECT` query — a cursor positioned BEFORE the first row initially, advanced one row at a time via `next()`, with typed accessor methods (`getString`, `getInt`, etc.) to read column values from the CURRENT row.

# 2. The Cursor Model

```
ResultSet rs = stmt.executeQuery("SELECT id, name, salary FROM employees");
// Cursor starts BEFORE the first row

while (rs.next()) { // moves the cursor to the NEXT row, returns false when exhausted
int id = rs.getInt("id"); // read by COLUMN NAME (readable, slightly slower)
int id2 = rs.getInt(1); // read by COLUMN INDEX (1-based, slightly faster)
String name = rs.getString("name");
double salary = rs.getDouble("salary");
}
```

# 3. `ResultSet` Types — Scrollability and Concurrency

```
Statement stmt = conn.createStatement(
ResultSet.TYPE_SCROLL_INSENSITIVE, // can move BACKWARD and to arbitrary positions
ResultSet.CONCUR_READ_ONLY // cannot update the database through this ResultSet
);
```

| Type Constant | Behavior |
|---|---|
| `TYPE_FORWARD_ONLY` (default) | Can only call `next()` — cannot go backward |
| `TYPE_SCROLL_INSENSITIVE` | Can move freely (`previous()`, `first()`, `last()`, `absolute(n)`), but doesn't reflect LIVE database changes made after the query ran |
| `TYPE_SCROLL_SENSITIVE` | Scrollable AND reflects live underlying data changes (less commonly supported/used) |
| `CONCUR_READ_ONLY` (default) | Cannot modify data through the `ResultSet` |
| `CONCUR_UPDATABLE` | Allows updating rows directly through the `ResultSet` (`updateString()`, then `updateRow()`) |

# 4. Key Methods

| Method | Purpose |
|---|---|
| `next()` | Advances to the next row, `false` if no more rows |
| `getXxx(columnLabel/Index)` | Retrieves a typed column value from the CURRENT row |
| `wasNull()` | Checks if the last retrieved column value was SQL `NULL` (important since primitives like `int` have no `null`) |
| `getMetaData()` | Returns `ResultSetMetaData` — column count, names, types (useful for generic/dynamic result processing) |

## 5. Edge Cases

| Case | Result |
|---|---|
| Calling `getXxx()` before the first `next()` call | `SQLException` — cursor is positioned BEFORE the first row initially |
| A SQL `NULL` column read via `getInt()` | Returns `0` (the primitive default) — you MUST call `wasNull()` immediately after to distinguish "actually zero" from "was NULL" |
| Reading a column that doesn't exist in the query | `SQLException` |
| Using a `ResultSet` after its parent `Statement`/`Connection` is closed | `SQLException` — a `ResultSet` is invalidated when its source Statement or Connection closes |

## 6. Common Mistakes

- Forgetting `getInt()` (and other primitive-returning getters) returns `0` for a SQL `NULL` value — must check `wasNull()` if distinguishing genuine zero from NULL matters.
- Using a default `TYPE_FORWARD_ONLY` `ResultSet` and then trying to call `previous()` — throws `SQLException`.
- Not closing the `ResultSet` (though try-with-resources on the `Statement`/`Connection` typically cascades this automatically in most JDBC driver implementations).

## 7. Frequently Asked Interview Questions

1 **What does `ResultSet` represent?** The tabular result of a `SELECT` query, accessed via a cursor advanced one row at a time.

2 **Where is the cursor positioned initially, before any `next()` call?** BEFORE the first row — you must call `next()` at least once before reading any column values.

3 **What does `getInt()` return for a SQL `NULL` value, and how do you detect that case?** It returns `0` (the primitive default); call `wasNull()` immediately afterward to check if the actual value was `NULL`.

4 **What's the difference between `TYPE_FORWARD_ONLY` and `TYPE_SCROLL_INSENSITIVE`?** `TYPE_FORWARD_ONLY` only allows moving forward via `next()`; `TYPE_SCROLL_INSENSITIVE` allows moving both forward AND backward, and jumping to arbitrary row positions.

---

# TOPIC 8: TRANSACTIONS

---

## 1. Introduction

**What is it?** A database transaction is a sequence of one or more SQL operations executed as a SINGLE, ATOMIC unit of work — either ALL operations succeed and are permanently saved (`commit`), or if ANY fails, ALL are undone (`rollback`), leaving the database exactly as it was before the transaction began.

**Why introduced**: Many real-world operations require multiple related database changes to happen together, consistently (e.g., transferring money: debit ONE account AND credit ANOTHER). Without transactions, a failure partway through (e.g., debiting succeeds but crediting fails due to a network error) would leave the database in a corrupted, inconsistent state — money would simply vanish.

## 2. The ACID Properties

| Property | Meaning |
|---|---|
| **Atomicity** | All operations in the transaction succeed together, or none do — no partial completion |
| **Consistency** | The database moves from one VALID state to another valid state, never violating defined constraints/rules |
| **Isolation** | Concurrent transactions don't interfere with each other's intermediate (uncommitted) state |
| **Durability** | Once committed, changes survive even a subsequent system crash |

## 3. Code Example — A Classic Bank Transfer

```
import java.sql.*;

public class TransactionDemo {
public static void transferMoney(Connection conn, int fromAccount, int toAccount, double amount)
throws SQLException {
try {
conn.setAutoCommit(false); // DISABLE auto-commit - start a manual transaction

try (PreparedStatement debit = conn.prepareStatement(
"UPDATE accounts SET balance = balance - ? WHERE id = ?")) {
debit.setDouble(1, amount);
debit.setInt(2, fromAccount);
debit.executeUpdate();
}

try (PreparedStatement credit = conn.prepareStatement(
"UPDATE accounts SET balance = balance + ? WHERE id = ?")) {
credit.setDouble(1, amount);
credit.setInt(2, toAccount);
credit.executeUpdate();
}

conn.commit(); // BOTH succeeded - make it PERMANENT
System.out.println("Transfer successful");

} catch (SQLException e) {
conn.rollback(); // EITHER failed - UNDO everything
System.out.println("Transfer failed, rolled back: " + e.getMessage());
throw e;
} finally {
conn.setAutoCommit(true); // restore default behavior for connection reuse
}
}
}
```

## 4. Transaction Isolation Levels

| Level | Prevents | Allows |
|---|---|---|
| `READ_UNCOMMITTED` | Nothing | Dirty reads, non-repeatable reads, phantom reads (lowest isolation, highest concurrency) |
| `READ_COMMITTED` | Dirty reads | Non-repeatable reads, phantom reads (common DEFAULT for many databases) |
| `REPEATABLE_READ` | Dirty + non-repeatable reads | Phantom reads |
| `SERIALIZABLE` | All of the above | Nothing (highest isolation, lowest concurrency — transactions behave as if executed one at a time) |

```
conn.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
```

## 5. Real Life Analogy

- **Bank**: A money transfer is the textbook transaction example — debiting one account and crediting another MUST happen together as one unit; if the system crashes after the debit but before the credit, the ENTIRE operation must roll back, or money would simply disappear from the system.

## 6. Edge Cases

| Case | Result |
|---|---|
| Forgetting `setAutoCommit(false)` | Every individual statement auto-commits immediately by default — no actual "transaction" grouping occurs, defeating the purpose |
| An exception AFTER `commit()` has already been called | Too late — the transaction is already permanently saved; rollback is no longer possible for that transaction |
| Two transactions modifying the SAME row concurrently | Isolation level determines exactly what each transaction can "see" of the other's uncommitted/committed changes, and whether they block each other |

## 7. Common Mistakes

- Forgetting to disable auto-commit before grouping multiple statements — each executes as its OWN separate mini-transaction, breaking atomicity.
- Not wrapping the transaction logic in a proper try-catch-rollback pattern, leaving the database in an inconsistent state if a mid-transaction failure occurs.
- Choosing an overly strict isolation level (`SERIALIZABLE`) by default without considering the real concurrency/performance trade-off it imposes.

## 8. Best Practices

- Always explicitly manage `setAutoCommit(false)` / `commit()` / `rollback()` for any multi-statement operation that must be atomic.
- Choose the LOWEST isolation level that still satisfies your application's correctness requirements — higher isolation generally means lower concurrency/throughput.
- In modern Spring-based applications, prefer the declarative `@Transactional` annotation (covered in the Spring Data JPA topic later) over manually managing `Connection` transaction state — same underlying concept, far less boilerplate.

## 9. Frequently Asked Interview Questions

1. **What is a database transaction?** A sequence of SQL operations executed as a single atomic unit — either all succeed (commit) or all are undone (rollback).
2. **What does ACID stand for?** Atomicity, Consistency, Isolation, Durability.
3. **What must you do before manually grouping multiple statements into one transaction in JDBC?** Call `conn.setAutoCommit(false)` — otherwise each statement auto-commits individually.
4. **What's the difference between `READ_COMMITTED` and `SERIALIZABLE` isolation levels?** `READ_COMMITTED` prevents dirty reads but still allows non-repeatable/phantom reads; `SERIALIZABLE` prevents ALL such anomalies by making transactions behave as if executed strictly one at a time, at the cost of concurrency/throughput.
5. **What's a real-world example requiring transactional atomicity?** A bank transfer — debiting one account and crediting another must succeed or fail TOGETHER, never partially.

---

# TOPIC 9: BATCH PROCESSING

---

## 1. Introduction

**What is it?** Batch processing groups multiple SQL statements together and sends them to the database in ONE network round-trip, dramatically reducing overhead compared to executing each statement individually — critical for bulk insert/update operations.

## 2. Code Example — Batch Insert with `PreparedStatement`

```
import java.sql.*;
import java.util.List;

public class BatchDemo {
public static void insertEmployees(Connection conn, List<String> names) throws SQLException {
String sql = "INSERT INTO employees (name) VALUES (?)";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
conn.setAutoCommit(false);

for (String name : names) {
pstmt.setString(1, name);
pstmt.addBatch(); // QUEUE this set of parameters, don't execute yet
}

int[] results = pstmt.executeBatch(); // sends ALL queued statements in ONE round-trip
conn.commit();
System.out.println("Inserted " + results.length + " rows");
}
}
}
```

## 3. Why Batching Matters — Performance Impact

```
WITHOUT BATCHING (1000 individual inserts):
1000 separate network round-trips to the database -> SLOW

WITH BATCHING (1000 inserts batched, e.g., in groups of 100):
Far fewer network round-trips -> MUCH faster, especially over higher-latency connections
```

## 4. Edge Cases

| Case | Result |
|---|---|
| One statement in a batch fails | Depending on the driver, may throw `BatchUpdateException`, which includes partial results for statements that succeeded BEFORE the failure |
| Extremely large batch size (millions of rows in one batch) | Can consume significant memory; best practice is to batch in reasonably-sized CHUNKS (e.g., 500-1000 at a time), executing and clearing periodically |

## 5. Best Practices

- Batch in reasonably-sized chunks rather than one enormous batch, to balance memory usage against round-trip reduction.
- Always disable auto-commit and manage the transaction explicitly around a batch operation.
- Handle `BatchUpdateException` properly to understand exactly which statements succeeded before a failure occurred.

## 6. Frequently Asked Interview Questions

1. **What problem does batch processing solve?** It reduces the number of network round-trips to the database by grouping multiple statements into one send operation, dramatically improving bulk operation performance.
2. **How do you queue statements for batch execution with `PreparedStatement`?** Call `addBatch()` after setting parameters for each set of values, then call `executeBatch()` once to send them all together.
3. **What exception can occur if one statement in a batch fails?** `BatchUpdateException`, which includes information about which statements succeeded before the failure.

---

# TOPIC 10: CONNECTION POOLING

---

## 1. Introduction

**What is it?** Connection pooling maintains a reusable, pre-established set of database `Connection` objects, handed out to application code on demand and RETURNED to the pool (not actually closed/destroyed) when no longer needed — avoiding the significant overhead of establishing a brand-new physical database connection for every single request.

**Why introduced**: Creating a raw database connection involves a real network handshake, authentication, and session setup on the database server — genuinely expensive (often tens of milliseconds). A busy web application handling thousands of requests per second CANNOT afford to pay this cost on every single request; pooling amortizes this cost by reusing a managed set of already-established connections.

## 2. How Connection Pooling Works

```
APPLICATION CODE CONNECTION POOL (e.g., HikariCP)
| +--------------------------+
|-- request a connection ----------->| [Conn1] [Conn2] [Conn3] | <- pre-established, idle connections
|<-- hands out an AVAILABLE Conn -----| [Conn4-in use] [Conn5] |
| +--------------------------+
| (application uses the connection,
| then calls .close())
|
|-- .close() called ---------------->| Connection is RETURNED to
| | the pool (NOT actually
| | destroyed!), ready for reuse
```

> **Critical insight**: Calling `.close()` on a POOLED connection does NOT actually close the underlying physical database connection — it's intercepted by the pool's connection WRAPPER, which returns the connection to the pool for the NEXT request to reuse. This is why the try-with-resources pattern (`try (Connection conn = ...)`) remains correct and important EVEN with pooling — it's what triggers the "return to pool" behavior.

## 3. HikariCP — The Modern Industry Standard

```
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost:3306/company_db");
config.setUsername("root");
config.setPassword("password");
config.setMaximumPoolSize(10); // max connections the pool will ever hold

HikariDataSource dataSource = new HikariDataSource(config);

try (Connection conn = dataSource.getConnection()) { // borrows from the pool
// use the connection normally
} // .close() returns it to the pool, NOT a real disconnect
```

> **Spring Boot uses HikariCP as its default connection pool** — when you configure `spring.datasource.*` properties in a Spring Boot application (covered later in this course), you're implicitly configuring HikariCP underneath.

## 4. Real Life Analogy

- **Restaurant**: A connection pool is like a fixed fleet of delivery scooters (connections) a restaurant chain owns — instead of BUYING a brand-new scooter (creating a fresh connection) for every single delivery order and scrapping it afterward, drivers simply RETURN the scooter to the depot after each delivery, ready for the next order to reuse immediately.

## 5. Key Pool Configuration Parameters

| Parameter | Purpose |
|---|---|
| `maximumPoolSize` | The absolute maximum number of connections the pool will ever create |
| `minimumIdle` | The minimum number of idle connections kept ready, even under low load |
| `connectionTimeout` | How long a request will wait for an available connection before giving up (throws an exception) |
| `idleTimeout` | How long an idle connection can sit before being closed/recycled |
| `maxLifetime` | Maximum lifetime of a connection before it's retired and replaced (even if still healthy) — helps avoid stale/long-lived connection issues |

## 6. Edge Cases

| Case | Result |
|---|---|
| All pool connections are currently in use, a new request comes in | Blocks/waits up to `connectionTimeout`, then throws an exception if no connection becomes available in time |
| An application forgets to `close()` (return) a borrowed connection | A genuine "connection leak" — that connection is never returned to the pool, eventually exhausting the pool's available connections entirely |
| Pool size set far too small for actual concurrent load | Requests queue up waiting for available connections, degrading application throughput/latency |
| Pool size set far too LARGE | Can overwhelm the DATABASE server itself, which has its own maximum connection limits |

## 7. Common Mistakes

- Not using try-with-resources (or otherwise reliably closing/returning) pooled connections — the single most common cause of connection pool exhaustion in production incidents.
- Sizing the pool arbitrarily large "just to be safe," without considering the DATABASE server's own connection limits, which are shared across potentially many application instances.
- Manually managing raw `DriverManager` connections in a high-throughput production application instead of using a proper pool.

## 8. Best Practices

- Always use a connection pool (HikariCP is the current industry-standard default, including in Spring Boot) in any real application beyond a trivial script.
- Size the pool deliberately based on actual expected concurrency and the database's own connection limits — not arbitrarily.
- Always release (close) borrowed connections reliably, via try-with-resources.

## 9. Production Usage

- Virtually every production Java web application uses connection pooling — it's not optional at any meaningful scale, given the real cost of raw connection establishment.
- Spring Boot auto-configures HikariCP by default whenever a `DataSource` is needed, requiring only simple `application.properties`/`application.yml` configuration (`spring.datasource.hikari.maximum-pool-size`, etc.) rather than manual pool setup code.

## 10. Frequently Asked Interview Questions

1 **What problem does connection pooling solve?** It avoids the significant overhead (network handshake, authentication) of establishing a brand-new physical database connection for every single request, by reusing a managed set of pre-established connections.

2 **What actually happens when you call `.close()` on a POOLED connection?** It's NOT actually closed/destroyed — the pool's connection wrapper intercepts the call and returns the connection to the pool, making it available for the next request to reuse.

3 **What is HikariCP?** The modern, high-performance, industry-standard Java connection pool implementation, used as Spring Boot's default connection pool.

4 **What happens if all pooled connections are in use and a new request needs one?** It waits (blocks) up to the configured `connectionTimeout`, throwing an exception if no connection becomes available within that time.

5 **What's the most common real-world cause of connection pool exhaustion?** Application code failing to properly close/return borrowed connections (a "connection leak"), often due to missing try-with-resources or an exception path that skips the close call.

---

## FINAL SUMMARY — JDBC COMPLETE

This concludes the **JDBC** section — 10 topics:

1 **JDBC Architecture** — the layered API/driver/database model enabling vendor-agnostic database access.

2 **Drivers** — Type 4 (Thin) drivers, modern auto-registration via SPI, JDBC URL format.

3 **Connection** — the session with the database, resource management, `try-with-resources`.

4 **Statement** — plain SQL execution, and the critical SQL Injection risk it exposes with concatenated user input.

5 **PreparedStatement** — the safe, performant, parameterized standard for virtually all real SQL execution.

6 **CallableStatement** — invoking stored procedures with IN/OUT/INOUT parameters.

7 **ResultSet** — the cursor-based model for reading query results, scrollability/updatability options.

8 **Transactions** — ACID properties, manual commit/rollback, isolation levels.

9 **Batch Processing** — grouping statements to minimize network round-trips for bulk operations.

10 **Connection Pooling** — HikariCP, why raw connection creation is too expensive for production, and what `.close()` really does on a pooled connection.

**Everything in this section forms the direct foundation for Hibernate/JPA and Spring Data JPA later in this course** — those frameworks automate and abstract exactly these JDBC mechanics (connections, statements, transactions, pooling) behind a higher-level, object-oriented API.

---

*(END OF JDBC MASTER NOTES — ready to proceed to Servlets whenever you say "Next Topic.")*